From Vibe Coding to Spec Coding

Agentic Programming for Compilation

Jian Weng

CEMSE, KAUST

Week 12, Session 0

Context

- I am teaching CS294Z this semester
- CS294Z: Agentic Programming for Compilation
- Goal: use agents to write and manage large codebases
- Compiler is just a case study for this flow
- Core question: how do we make AI-assisted programming persistent?

Persistent

- This is a pun, especially under the nature of Computer Science.
- In almost **ALL** CS class, we learn:
  - How to use a tool
  - How this tool works
- Persistence is about how:
  - AI agent run persistently - a long enough session
  - AI-generated code persistent - maintainable over time

What Is Vibe Coding?

Typical pattern:

1. Start an AI agent session
2. Tell the agent what to build or change
3. Wait for generated code
4. Interrupt and redirect when it goes wrong
5. Sometimes let the agent run while you do something else
    - ~~You even forget the agent is running when slacking off~~

What do we want?

- An AI agent that runs long enough to do something useful
  - Long enough to amortize my time and attention overhead
  - Useful enough to give meaningful results

What Does "Vibe" Mean?

- Vibe ~~is not viberation~~
- Vibe means atmosphere or feeling
- But be careful about your feelings

✅ You should be specific to what you want

- It is sort of ok not to know the exact implementation path

❌ You cannot just say "make it better" or "fix the bug"

When Vibe Coding Works

- Great for:
  - Short-term tasks
  - One-off scripts
  - Single-file quick PoC
- **Not** great for:
  - Long-term maintenance is not the goal

Where Vibe Coding Breaks Down

- Multi-file systems
- Long-term maintenance
- Shared codebases
- Complicated codebase with many dependent stages

For these cases, use spec coding.

What Is Spec Coding?

- Spec means **spec**ification and standard
- Describe how the codebase should be maintained
- Define expectations before asking the agent to implement
- Make documentation, testing, and functionality part of the workflow
- Manage the agent like a team member, not just a code generator

AI Gives You a Team of "You"

Before:

- You only have yourself
- You maintain the codebase by developing the kernels of implementation.

Now:

- Many agent sessions may touch the codebase
- Project rules must be written down
- Consistency depends on shared specs

The Spec Maintenance

Documentation:

- Helps long-term maintenance
- Gives the LLM natural-language context of understanding

Testing:

- Catches regressions quickly
- Makes agent iteration safer

Functionality:

- Still the core, but only part of the whole codebase

A Practical LoC Rule

- LoC - Lines of Code
- This is not from any textbook or open data, but my experience
- Document, test, and functionality is around 1:1:1 LoC
- When asking AI to estimate the amount of work
  - **DO NOT** ask for ETA as they write code tooooooooo fast
  - **DO** ask for LoC estimation for each aspect

Without writing code, how can I trust LoC?

- Translator: for which transformer-based LLMs were designed.
- It originally translates language A to language B with different length.
- Beyond translating languages, it translates questions to answers.
    - Just like if CNN recoginizes cats, it should recognize dogs.
- "Translate" the current code state to LoC estimation is simple.

Test-driven Development (TDD)

- When you ask agent to implement something
- First write the documents of designing the interface
- Then write the test cases for the intended behavior calling the interface
- Finally, let the agent iterate over until all the tests pass

The goal is not perfect correctness.

The goal is stable evolution without degradation.

First Tool: `AGENTS.md` / `CLAUDE.md`

- This TDD flow is a fixed flow, why don't we solidate it?
- Encode rules/specs in AI's mind.
- `AGENTS.md` / `CLAUDE.md` are the highest priority of the agent's context.
- When entering a folder, that file is surely read.
- Write done these rules to ask AI to develop the code.

Architect the Docs

Question:

- if your manager does not read the code, how do they know the project state?

Answer:

- keep a `docs` folder at the repository root
- track architecture, design decisions, roadmap, and progress
- simple Markdown files are enough
- mention `docs` in `AGENTS.md` / `CLAUDE.md`

Architect the Docs (CONT'D)

- Maybe you also would like a `xxxx.md` file right next to the source code
- Which docs
  - the interface exposed to other files.
  - the implementation details
  - the internal helper functions

Besides `AGENTS.md`

Other configuration surfaces:

- `rules`: task-specific code of conduct
- `prompts`: reusable prompt templates with arguments
- `skills`: specifications for unfamiliar tools and workflows

Each one narrows the agent's behavior in a different way.

`rules`

- weaker than `AGENTS.md` / `CLAUDE.md`
- more specific to the current task
- selected by a task description or context
- useful for local conventions that should not be global
- similar to a code of conduct for a task family

`prompts`

- reusable prompt templates
- similar to functions for LLM interaction
- use string substitution to fill in details
- good for frequent workflows
- reduce repeated ad hoc instructions

`skills`

https://agentskills.io/

- not just macros or implicit functions
- better viewed as few-shot specifications
- teach the agent how to use a tool or workflow
- especially useful when the tool is:
  - custom
  - new
  - internal
  - not well represented in model training data

Specifications as Few-Shot Learning

- common tools are already familiar to LLMs
- examples: `git`, `python`, `gcc`
- custom tools need explicit guidance
- a skill gives the model examples and constraints
- the agent can then apply familiar command-line reasoning to an unfamiliar tool

What Is Few-Shot Learning?

- LLMs are good at learning from references in context
- a reference implementation becomes a useful example
- this is one reason rewrite tasks often work well
- A joke here: Why Rust is good for LLM programming?

Daily Workflow Example

- a Google Workspace CLI manages email, Drive, calendar, and TODOs
- the tool is too new for the model to know by default
- official skills provide the missing specification
- the agent can then draft emails, create reimbursement requests, and handle chores
- I use this to manage my daily work tasks at KAUST
- https://github.com/googleworkspace/cli/tree/main/skills

This is vibe coding for short-term personal tasks.

Vibe Coding vs Spec Coding

Vibe coding:

- short-term
- exploratory
- outcome-driven
- low maintenance pressure

Spec coding:

- long-term
- systematic
- standards-driven

Summary

- We had two goals of persistence
  - The AI-generated code should be maintainable now
  - How can we make the AI agent run persistently?

The Ralph Loop

https://claude.com/plugins/ralph-loop

- To some extent, we already took advantage of AI's persistence in vibe coding
  - If the test fails, keep coding.
- We can do this even more aggressively with spec coding
  - If my goal is not achieved yet, keep coding.
- Question: Where does this goal come from?

Plan first, code later

I spoilt a little bit before:

- ~~Estimate the LoC before coding~~
- Plan first before coding - estimating LoC is a part of planning
- Tell AI specifically what you want to make a plan **w/o impl**
- The plan is not mature at once, you can repeat the loop until it is good

What is a good plan?

- No design decisions is made when implementing the plan

  ❌ Audit the code base to find the case to modify

  ✅ Audit is already done during planning

  - `/path/to/file1:line` should be fixed
  - `/path/to/file2:line` should be refactored
  - `/path/to/file3:line` should be extended
- If the design decision is unclear/non-obvious
  - AI should interact with you

Execute a plan

https://www.anthropic.com/engineering/building-c-compiler

Once the plan is converged, write a shell script to run it.

```
while [ true ]; do
  if [ all_plan_items_cleared ]; then
    break
  fi
  codex exec "run the plan"
  codex exec "stage and commit"
  codex exec "simplify and refactor the codebase"
  codex exec "stage and commit"
done
```

Conclusion

- vibe coding and spec coding are both useful
- they target different areas of applicability
- use vibe coding when the task is small and short-lived
- use spec coding when the codebase must survive future changes
- for compiler projects, spec coding should be the default